

BI Woche 3 Aufgaben

R-Einführung für Studierende (BI-Kontext) –

10 Übungen mit Lösungen & Beispielen

Diese Übungen orientieren sich am Inhalt der Vorlesungsfolien **Business Intelligence / Data Warehouse / MySQL & R / R Recap** (u.a. Datentypen in R, Indexing, Data Frames, Factors, logische Operationen, sowie MySQL-Verbindung aus R). :contentReference[oaicite:0]{index=0}

Übung 1 – Vektoren anlegen & grundlegende Operationen

Ziel: Einen numerischen Vektor erstellen, Elemente auslesen und einfache Kennzahlen berechnen.

Aufgabe:

1. Erstelle einen Vektor `v` mit den Zahlen 1 bis 10.
2. Gib das 10. Element aus.
3. Berechne Mittelwert, Summe, Minimum und Maximum.

Lösung (Beispielcode):

```
v <- c(1,2,3,4,5,6,7,8,9,10)

v[10]           # 10. Element
mean(v)         # Mittelwert
sum(v)          # Summe
min(v)          # Minimum
max(v)          # Maximum
```

Übung 2 – Indexing: Werte ersetzen (wie im Recap)

Ziel: Zugriff per Index und Änderung von Elementen.

Aufgabe:

1. Verwende wieder `v <- 1:10`.
2. Setze das 10. Element auf 0.
3. Gib `v` aus.

Lösung (Beispielcode):

```
v <- 1:10  
v[10] <- 0  
v
```

Übung 3 – Logische Indizes: Filtern & Ersetzen

Ziel: Mit Bedingungen Teilmengen auswählen und Werte ersetzen.

Aufgabe:

1. Erstelle `v <- 1:10`.
2. Erzeuge einen logischen Vektor `v > 5`.
3. Ersetze alle Werte `> 5` durch 10.

Lösung (Beispielcode):

```
v <- 1:10  
  
v > 5          # logischer Vektor (TRUE/FALSE)  
v[v > 5] <- 10 # Ersetzen mit logischem Index  
v
```

Übung 4 – Matrix erstellen & Elemente über [row, col] ändern

Ziel: Matrix aus einem Vektor bauen und per zweidimensionalem Indexing bearbeiten.

Aufgabe:

1. Erstelle `v <- 1:10`.
2. Baue eine Matrix mit 5 Zeilen und 2 Spalten (zeilenweise befüllen).
3. Setze Element `[1,1]` auf 0 und `[1,2]` auf 100.

Lösung (Beispielcode):

```
v <- 1:10  
m <- matrix(v, ncol = 2, nrow = 5, byrow = TRUE)  
  
m[1,1] <- 0  
m[1,2] <- 100  
m
```

Übung 5 – Warum Data Frames? (gemischte Datentypen)

Ziel: Verstehen, dass Matrizen nur *einen* Datentyp enthalten – Data Frames sind flexibler.

Aufgabe:

1. Erstelle eine Matrix aus Zahlen (wie oben).
2. Versuche, einen String in ein Matrixfeld zu schreiben.
3. Erkläre (kurz), was passiert ist.
4. Erstelle stattdessen einen Data Frame, der Zahlen und Text enthält.

Lösung (Beispielcode):

```
v <- 1:10
m <- matrix(v, ncol = 2, nrow = 5, byrow = TRUE)

m[1,2] <- "Word"
m
# Hinweis: Die Matrix wird typischerweise zu "character" umgewandelt, weil
# Matrizen nur 1 Datentyp haben.

df <- data.frame(
  id = 1:5,
  label = c("A", "B", "C", "D", "E")
)
df
str(df)
```

Übung 6 – Factors: Level definieren & typische Falle

Ziel: Factors anlegen und verstehen, warum “neue” Kategorien Probleme machen können.

Aufgabe:

1. Erstelle einen Factor `f` aus `c("R", "Python", "R", "R", "Python")` mit Levels `c("R", "Python")`.
2. Baue einen Data Frame daraus.
3. Versuche, einen Eintrag `"NotPython"` zu setzen (und beobachte das Ergebnis).

Lösung (Beispielcode):

```
f <- factor(c("R", "Python", "R", "R", "Python"), levels = c("R", "Python"))
df <- data.frame(Number = 1:5, Factor = f)

df$Factor[2] <- "NotPython" # typischerweise Warning + NA
df
```

Übung 7 – Factors korrekt “updaten”

Ziel: Levels erweitern bzw. neu setzen, damit neue Kategorien erlaubt sind.

Aufgabe:

1. Starte mit dem Data Frame aus Übung 6.
2. Erweitere die Levels so, dass `"NotPython"` erlaubt ist.
3. Setze dann bei allen `"R"`-Einträgen den Wert auf `"NotPython"`.

Lösung (Beispielcode):

```
f <- factor(c("R", "Python", "R", "R", "Python"), levels = c("R", "Python"))
df <- data.frame(Number = 1:5, Factor = f)

# Levels erweitern:
df$Factor <- factor(df$Factor, levels = c("R", "Python", "NotPython"))

# Ersetzen:
df$Factor[df$Factor == "R"] <- "NotPython"
df
```

Übung 8 – Mini-ETL in R: “Extract–Transform–Load” als Konzept

Ziel: ETL-Idee praktisch nachvollziehen (vereinfacht in R; kein echtes DW nötig).

Aufgabe:

1. **Extract:** Erstelle zwei kleine Data Frames: `sales_a` und `sales_b` (unterschiedliche Spaltennamen!).
2. **Transform:** Vereinheitliche Spaltennamen, entferne Duplikate, fülle fehlende Werte.
3. **Load:** Füge beide Quellen zusammen in `sales_all` und berechne Umsatz je Produkt.

Lösung (Beispielcode):

```
# Extract (zwei Quellen)
sales_a <- data.frame(
  product = c("Apfel", "Brot", "Milch", "Milch"),
  amount  = c(10, 5, NA, 3),
  price   = c(1.2, 2.5, 1.8, 1.8)
)

sales_b <- data.frame(
  Produkt = c("Apfel", "Käse"),
  Menge   = c(7, 2),
  Preis    = c(1.2, 4.0)
)

# Transform: Spalten angleichen
names(sales_b) <- c("product", "amount", "price")

# fehlende Werte: NA in amount z.B. als 0 interpretieren
sales_a$amount[is.na(sales_a$amount)] <- 0

# Duplikate entfernen (optional: hier über komplette Zeilen)
sales_a <- unique(sales_a)

# Load: zusammenführen
sales_all <- rbind(sales_a, sales_b)

# Kennzahl: Umsatz pro Zeile und aggregiert pro Produkt
sales_all$revenue <- sales_all$amount * sales_all$price
```

```
revenue_by_product <- aggregate(revenue ~ product, data = sales_all, sum)
sales_all
revenue_by_product
```

Übung 9 – OLTP vs. OLAP-Denke in R: “Transaktionen” vs. “Analyse”

Ziel: Unterschied gedanklich nachvollziehen: Transaktionsdaten (Detail) → Analyse (Aggregation).

Aufgabe:

1. Simuliere “OLTP” Transaktionsdaten: einzelne Verkäufe mit Datum, Produkt, Menge, Preis.
2. Erstelle eine “OLAP”-ähnliche Auswertung: Umsatz pro Monat und Produkt.

Lösung (Beispielcode):

```
set.seed(42)

oltp <- data.frame(
  date      = as.Date("2026-01-01") + sample(0:59, 50, replace = TRUE),
  product   = sample(c("Apfel", "Brot", "Milch"), 50, replace = TRUE),
  amount    = sample(1:5, 50, replace = TRUE),
  price     = c(Apfel = 1.2, Brot = 2.5, Milch = 1.8)
[sample(c("Apfel", "Brot", "Milch"), 50, replace = TRUE)]
)

# Transform: Monat ableiten + Umsatz
oltp$month <- format(oltp$date, "%Y-%m")
oltp$revenue <- oltp$amount * oltp$price

# "OLAP": Aggregation
olap <- aggregate(revenue ~ month + product, data = oltp, sum)
olap[order(olap$month, olap$product), ]
```

Übung 10 – MySQL aus R verbinden (Template + Beispielabfrage)

Ziel: Grundmuster “Client–Server”: Verbindung herstellen, Query ausführen, Resultat als Data Frame nutzen.

Hinweis: Dafür muss ein MySQL-Server laufen und Zugangsdaten müssen stimmen. In den Folien wird genau dieses Prinzip gezeigt (Konsole: `mysql -u root -p`, sowie Verbindung aus R).

Aufgabe:

1. Installiere (falls nötig) ein passendes DB-Paket.
2. Baue eine Verbindung auf.
3. Führe eine einfache SQL-Abfrage aus (z.B. “zeige Tabellen” oder “SELECT ... LIMIT

10”).

4. Schließe die Verbindung.

Lösung (Beispielcode mit DBI + RMySQL):

```
# Installation (einmalig):  
# install.packages("DBI")  
# install.packages("RMySQL")  
  
library(DBI)  
library(RMySQL)  
  
con <- dbConnect(  
  MySQL(),  
  host = "localhost",  
  user = "root",  
  password = "DEIN_PASSWORT",  
  dbname = "DEINE_DATENBANK",  
  port = 3306  
)  
  
# Beispiel 1: Tabellen anzeigen  
tables <- dbListTables(con)  
tables  
  
# Beispiel 2: Query (ersetze table_name)  
# result <- dbGetQuery(con, "SELECT * FROM table_name LIMIT 10;")  
# head(result)  
  
dbDisconnect(con)
```

Bonus (optional): Wenn du eine “Flughafen-Datenbank” wie in den Folien verwendest, suche dir 1–2 Tabellen aus und mache eine kleine OLAP-Auswertung (z.B. Anzahl Flüge pro Airline, pro Monat, etc.).

Mini-Checkliste (für Abgabe/Review)

- Läuft der Code ohne Fehler (bis auf Übung 10 ohne echte DB)?
 - Sind Objekte sinnvoll benannt (v, m, df, oltp, olap)?
 - Werden Data Frames statt Matrizen genutzt, wenn Datentypen gemischt sind?
 - Wurden Factors korrekt behandelt (Levels!)?
 - Bei “Analyse”: wurden Kennzahlen/ Aggregationen (z.B. aggregate) verwendet?
-